



Lab 04: Manage Terraform State

Build Infrastructure with Terraform | GSP752

Overview

Configured local and Cloud Storage backends for Terraform state management. Refreshed state to detect infrastructure drift. Imported an existing Docker container into Terraform and managed it going forward.

Objectives

- Create a local backend
- Create a Cloud Storage backend
- Refresh Terraform state
- Import existing infrastructure into Terraform
- Manage imported configuration with Terraform

Why Terraform State Matters

Purpose	Description
Mapping	Tracks which real-world resources match which config blocks
Metadata	Stores resource dependencies for correct destroy ordering
Performance	Caches attribute values to avoid querying all resources on every run
Locking	Prevents concurrent changes from corrupting state in team environments

Task 1 - Work with Backends

Configured a local backend then migrated to a Cloud Storage backend.

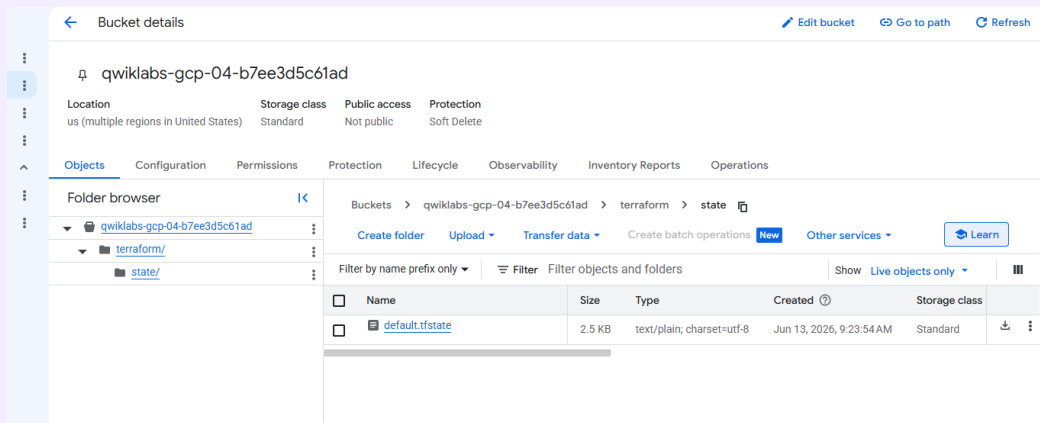
Local backend configuration:

```
terraform {
  backend "local" {
    path = "terraform/state/terraform.tfstate"
  }
}
```

Cloud Storage backend configuration:

```
terraform {
  backend "gcs" {
    bucket = "[BUCKET_NAME]"
    prefix = "terraform/state"
  }
}
```

```
terraform init -migrate-state
```



Refreshed state after manually adding a label to the bucket in the console.

```
terraform refresh
terraform show
```

```
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
student_01_bcb192601222@cloudshell:~ (qwiklabs-gcp-04-b7ee3d5c61ad) $ terraform refresh
google_storage_bucket.test-bucket-for-state: Refreshing state... [id=qwiklabs-gcp-04-b7ee3d5c61ad]
student_01_bcb192601222@cloudshell:~ (qwiklabs-gcp-04-b7ee3d5c61ad) $ terraform show
# google_storage_bucket.test-bucket-for-state:
resource "google_storage_bucket" "test-bucket-for-state" {
  default_event_based_hold = false
  deletion_policy          = "DELETE"
  effective_labels         = {
    "goog-terraform-provisioned" = "true"
    "key"                       = "value"
  }
  enable_object_retention = false
  force_destroy           = false
  id                      = "qwiklabs-gcp-04-b7ee3d5c61ad"
  labels                  = {}
  location                 = "US"
  name                    = "qwiklabs-gcp-04-b7ee3d5c61ad"
  project                 = "qwiklabs-gcp-04-b7ee3d5c61ad"
  project_number          = 110114931613
  public_access_prevention = "inherited"
  requester_pays          = false
  rpo                     = "DEFAULT"
  self_link               = "https://www.googleapis.com/storage/v1/b/qwiklabs-gcp-04-b7ee3d5c61ad"
  storage_class            = "STANDARD"
  terraform_labels        = {
    "goog-terraform-provisioned" = "true"
  }
  time_created            = "2026-06-13T13:21:59.698Z"
  uniform_bucket_level_access = true
  updated                 = "2026-06-13T13:26:18.512Z"
  url                     = "gs://qwiklabs-gcp-04-b7ee3d5c61ad"

  hierarchical_namespace {
    enabled = false
  }

  soft_delete_policy {
    effective_time         = "2026-06-13T13:21:59.698Z"
    retention_duration_seconds = 604800
  }
}
student_01_bcb192601222@cloudshell:~ (qwiklabs-gcp-04-b7ee3d5c61ad) $
```

Task 2 - Import Existing Infrastructure

Created a Docker container manually then imported it into Terraform state.

```
docker run --name hashicorp-learn --detach --publish 8080:80 nginx:latest
terraform import docker_container.web $(docker inspect -f {{.ID}} hashicorp-learn)
terraform show -no-color > docker.tf
```

Cleaned up docker.tf to only include required attributes and added a docker_image resource.

```
resource "docker_image" "nginx" {
  name = "nginx:latest"
}
resource "docker_container" "web" {
  image = docker_image.nginx.image_id
  name = "hashicorp-learn"
  ports { external = 8080 internal = 80 ip = "0.0.0.0" protocol = "tcp" }
}
```

[illegible]

What I Learned

- Terraform state maps real-world resources to configuration blocks
- Remote backends like GCS enable team collaboration with state locking
- terraform refresh detects drift between state and real infrastructure
- terraform import brings existing resources under Terraform management
- After import you must manually write the configuration to match the imported resource
- terraform show -no-color is useful for generating a starting configuration from state

Resources

Terraform State Docs: <https://developer.hashicorp.com/terraform/language/state> | Lab ID: GSP752